Name of Student Roll No.	Experiment No. 1	Date:

Tiltle: 4 Bit ALU for addition, subtraction, AND, NAND, XOR, XNOR, OR, & ALU pass.

Pre-lab Requisites:

1. Knowledge of Logic Gates
2. Details of logic Gates with truth table
3. Number system and its representation
4. Arithmetic and logical operation of binary numbers.

Objectives:

1. To write VHDL code for 4 Bit ALU
2. Simulate with test bench
3. Synthesis, implementation on FPGA

Equipments:

Sr. No.	Description	Specification
1	System	Processor - 2GHz or Higher, RAM – 2GB or Higher, OS – Windows 10 (32 Bit with SP1 or Higher)
2.	Xilinx	10.1
3.	Spartan evaluation board	Spartan 6 : xc6slx
4.	SMPS	

Theory:

Arithmetic & Logical Unit

An **arithmetic logic unit** (**ALU**) is a combinational digital electronic circuit that performs arithmetic and bitwise operations on integer binary numbers. An ALU is a fundamental building block of many types of computing circuits, including the central processing unit (CPU) of computers. The inputs to an ALU are the data to be operated on, called operands, and a code indicating the operation to be performed; the ALU's output is the result of the performed operation. In many designs, the ALU also has status inputs or outputs, or both, which convey information about a previous operation or the current operation, respectively, between the ALU and external status registers.

The system block diagram for a 4-bit ALU is shown in figure 1.1. ALU is a combinational circuit that performs logic and arithmetic micro-operations on a pair of n- bit operands (ex. A[3:0] and B[3:0]). The operations performed by an ALU are controlled by set of functions-select inputs. In this design a 4-bit ALU with 3 functions select (S0 S1 S2) inputs.

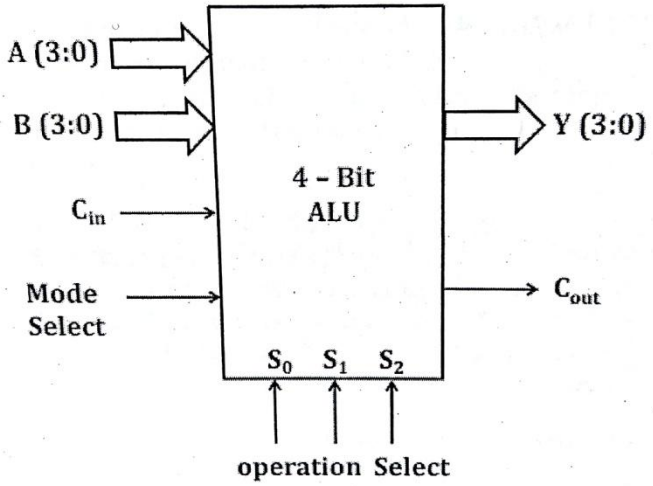


Figure 1.1: ALU Block Diagram

The inputs to the ALU are the data to be operated on (called operand) and a code from the control unit indicating which operation to be performed. In many designs the ALU also takes or generates as inputs and outputs a set of condition codes from or to the registers. These codes are used to indicate cases such as carry-in or carryout, overflow, divide-by-zero, etc.

A simple arithmetic and logic unit does AND, OR, XOR, NOT, addition. Most ALU can perform operations: integer arithmetic operations such as addition, substraction, multiplication, division.

Arithmetic operations

- *Add*: A and B are summed and the sum appears at Y and carry-out.
- *Add with carry*: A, B and carry-in are summed and the sum appears at Y and carry-out.
- *Subtract*: B is subtracted from A (or vice versa) and the difference appears at Y and carry-out. For this function, carry-out is effectively a "borrow" indicator. This operation may also be used to compare the magnitudes of A and B; in such cases the Y output may be ignored by the processor, which is only interested in the status bits (particularly zero and negative) that result from the operation.
- *Subtract with borrow*: B is subtracted from A (or vice versa) with borrow (carry-in) and the difference appears at Y and carry-out (borrow out).
- *Two's complement (negate)*: A (or B) is subtracted from zero and the difference appears at Y.
- *Increment*: A (or B) is increased by one and the resulting value appears at Y.
- *Decrement*: A (or B) is decreased by one and the resulting value appears at Y.
- *Pass through*: all bits of A (or B) appear unmodified at Y. This operation is typically used to determine the parity of the operand or whether it is zero or negative, or to load the operand into a processor register.

Bitwise logical operations

- *AND*: the bitwise AND of A and B appears at Y.
- *OR*: the bitwise OR of A and B appears at Y.
- *Exclusive-OR*: the bitwise XOR of A and B appears at Y.
- *Ones' complement*: all bits of A (or B) are inverted and appear at Y.

Procedure:

1. Open Xilinx ISE → File → New Project.
2. Enter the Project Name and select VHDL as the preferred language.
3. Select the required FPGA device, package, and speed grade.
4. Create the VHDL source file (.vhd).
5. Write your VHDL design code.
6. Right-click the design file → New Source → select VHDL Test Bench.
7. Enter the Test Bench name and select the design entity to be tested.
8. ISE automatically creates the VHDL test bench with the required component declaration and port connections.
9. Edit the test bench and provide input values and time delays.
10. Save the test bench.
11. In the Processes window, select ISim Simulator → Simulate Behavioral Model.
12. ISim opens the waveform window.
13. Select the required input and output signals.
14. Run the simulation for the desired duration.
15. Observe the generated input and output waveforms to verify the functionality of the VHDL design.

VHDL code:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity ALU is
Port ( inp_a : in signed(3 downto 0);
inp_b : in signed(3 downto 0);
sel : in STD_LOGIC_VECTOR (2 downto 0);
out_alu : out signed(3 downto 0));
end ALU;
architecture Behavioral of ALU is
begin
process(inp_a, inp_b, sel)
begin
case sel is
when "000" =>
out_alu<= inp_a + inp_b; --addition
when "001" =>
out_alu<= inp_a - inp_b; --subtraction
when "010" =>
out_alu<= inp_a - 1; --sub 1
when "011" =>
out_alu<= inp_a + 1; --add 1
when "100" =>
out_alu<= inp_a and inp_b; --AND gate
when "101" =>
out_alu<= inp_a or inp_b; --OR gate
when "110" =>
out_alu<= not inp_a ; --NOT gate
```

```
when "111" =>
out_alu<= inp_a xor inp_b; --XOR gate
when others =>
NULL;
end case;
end process;
end Behavioral;
```

TestBench Code:

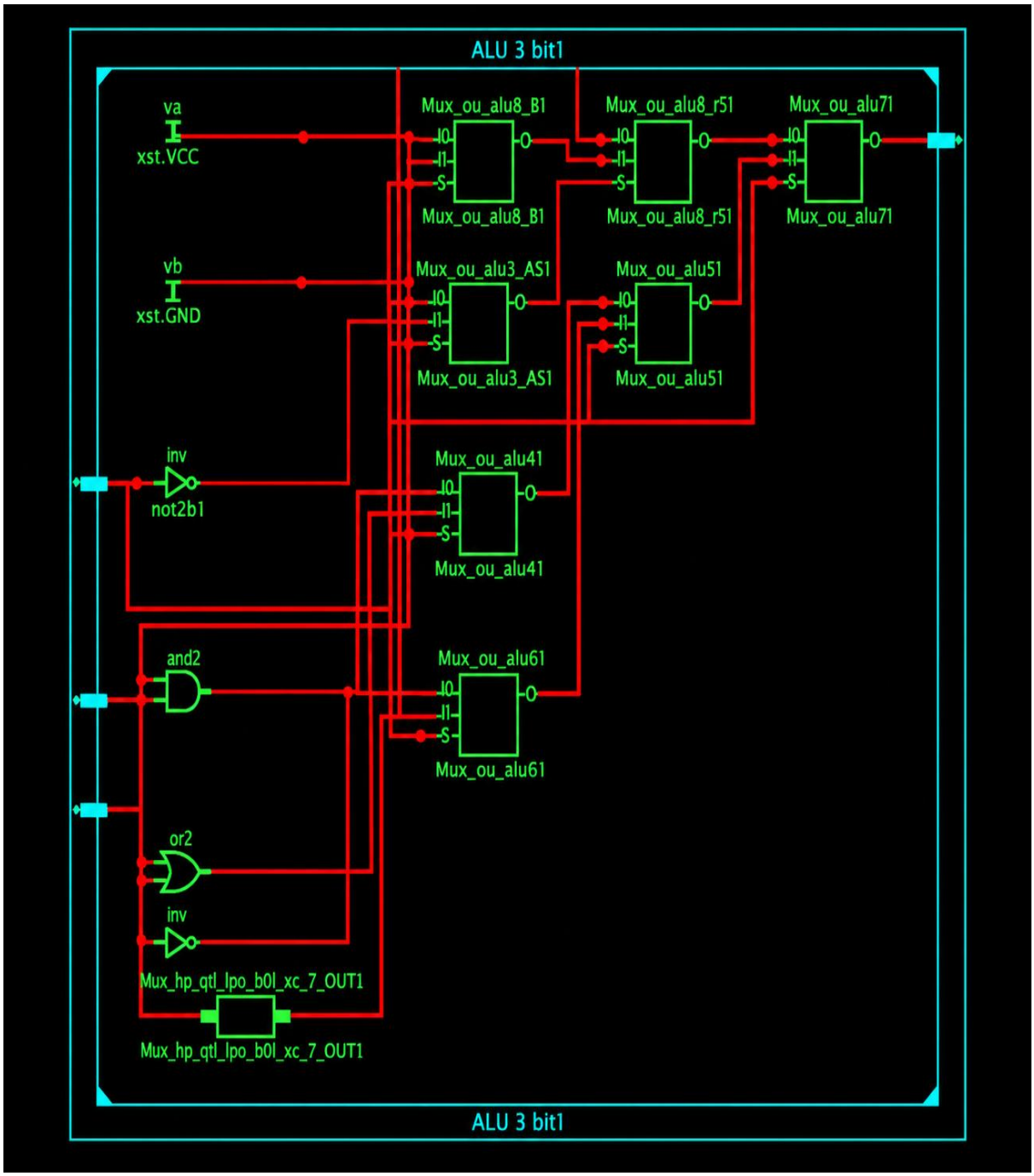
```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;
ENTITY TB_ALU IS
END TB_ALU;
ARCHITECTURE behavior OF TB_ALU IS
-- Component Declaration for the Unit Under Test (UUT)
COMPONENT ALU
PORT(
inp_a : IN signed(3 downto 0);
inp_b : IN signed(3 downto 0);
sel : IN std_logic_vector(2 downto 0);
out_alu : OUT signed(3 downto 0)
);
END COMPONENT;
signal inp_a : signed(3 downto 0) := (others => '0');
signal inp_b : signed(3 downto 0) := (others => '0');
signal sel : std_logic_vector(2 downto 0) := (others => '0');
signal out_alu : signed(3 downto 0);
-- No clocks detected in port list. Replace <clock> below with
-- appropriate port name
BEGIN
-- Instantiate the Unit Under Test (UUT)
uut: ALU PORT MAP (
inp_a => inp_a,
inp_b => inp_b,
sel => sel,
out_alu => out_alu
);
-- Stimulus process
stim_proc: process
begin
-- hold reset state for 100 ns.
wait for 100 ns;
-- insert stimulus here
inp_a <= "1001";
inp_b <= "1111";
sel <= "000";
wait for 100 ns;
```

```
sel <= "001";
wait for 100 ns;
sel <= "010";
wait for 100 ns;
sel <= "011";
wait for 100 ns;
inp_a <= "0101";
inp_b <= "0110";
sel <= "100";
wait for 100 ns;
sel <= "101";
wait for 100 ns;
sel <= "110";
wait for 100 ns;
sel <= "111";
wait;
end process;
END;
```

Constraint UCF File:

```
NET "inp_a<0>" LOC = "p6"; #SW0
NET "inp_a<1>" LOC = "p58"; #SW1
NET "inp_a<2>" LOC = "p64";#SW2
NET "inp_a<3>" LOC = "p75";#SW3
NET "inp_b<0>" LOC = "p8";
NET "inp_b<1>" LOC = "p61";
NET "inp_b<2>" LOC = "p67";
NET "inp_b<3>" LOC = "p116";
NET "sel<0>" LOC = "p141";
NET "sel<1>" LOC = "p137";
NET "sel<2>" LOC = "p131";
NET "out_alu<0>" LOC = "p7"; #LI8
NET "out_alu<1>" LOC = "p62";#LI7
NET "out_alu<2>" LOC = "p16";
NET "out_alu<3>" LOC = "p2";
```

Output: Schematic of an Arithmetic Logic Unit (ALU):



Implementation of Digital Logic Circuits on FPGA Development Board:



Conclusion:

Checked By:

Name of Subject Teacher: **Mr.P.D.Lokhande**	Sign with Date